

- **Họ tên:** Hồ Tuấn Huy
- **MSSV:** 20225856

Báo cáo kết quả bài tập Tạo bảng ký hiệu

I. Kết quả thực hiện với các ví dụ

1. *Kết quả thực hiện example1.kpl*

- Mã nguồn của example1.kpl:

```
tests > example1.kpl

1  PROGRAM EXAMPLE1;
2  BEGIN
3  END. (* Example 1 *)
```

- Kết quả thực hiện:

```
Problems  Output  Debug Console  Terminal  Ports  Spell Checker 21
● → SymTab_Incompleted ./symtab tests/example1.kpl
   Program EXAMPLE1
○ → SymTab_Incompleted
```

- **Giải thích kết quả thực hiện:** File example1.kpl chỉ có khai báo chương trình và một khối lệnh rỗng, không có hằng số, biến, kiểu dữ liệu, hàm hay thủ tục. Quá trình xử lý:
 - Khởi tạo symbol table: Hệ thống khởi tạo bảng ký hiệu và tạo các hàm/thủ tục built-in (như READC, READI, WRITEI, WRITEC, WRITELN). Các đối tượng built-in này không được in ra trong output cuối cùng.
 - Tạo đối tượng chương trình: Khi parser gặp PROGRAM EXAMPLE1;, nó gọi hàm createProgramObject("EXAMPLE1") để tạo đối tượng chương trình và đưa vào symbol table.
 - Khối lệnh rỗng: Phần thân BEGIN ... END. rỗng nên không có thêm đối tượng nào được khai báo trong scope của chương trình.
 - Kết quả in ra: Khi in symbol table, chương trình chỉ hiển thị các đối tượng do người dùng khai báo, không bao

gồm các built-in functions. Do đó, output chỉ hiển thị tên chương trình EXAMPLE1 mà không có biến, hàm hay thủ tục nào khác.

2. Kết quả thực hiện example2.kpl

- Mã nguồn của example2.kpl:

```
tests > example2.kpl
1  PROGRAM EXAMPLE2; (* Factorial *)
2
3  VAR N : INTEGER;
4
5  FUNCTION F(N : INTEGER) : INTEGER;
6  BEGIN
7      IF N = 0 THEN F := 1 ELSE F := N * F (N - 1);
8  END;
9
10 BEGIN
11     FOR N := 1 TO 7 DO
12     BEGIN
13         CALL WRITELN;
14         CALL WRITEI(F(n));
15     END;
16 END. (* Factorial *)
```

- Kết quả thực hiện:

```
● → SymTab_Incompleted ./symtab tests/example2.kpl
Program EXAMPLE2
  Var N : Int
  Function F : Int
    Param N : Int

○ → SymTab_Incompleted
```

- Giải thích kết quả thực hiện:

- Khi parser xử lý file example2.kpl, nó gặp các khai báo theo thứ tự:
 - Khai báo chương trình: PROGRAM EXAMPLE2; => Parser tạo đối tượng Program tên EXAMPLE2, đây là đối tượng gốc và tạo scope chính của chương trình.
 - Khai báo biến toàn cục: VAR N : INTEGER; => Parser tạo đối tượng Variable tên N kiểu Int và đưa vào global scope của chương trình.

- Khai báo hàm: FUNCTION F(N : INTEGER) : INTEGER;
- Khi xử lý khai báo hàm, parser thực hiện:
 - Tạo Function Object tên F với return type là Int, thuộc scope của chương trình.
 - Tạo Parameter Object tên N kiểu Int, được thêm vào danh sách tham số của hàm F và scope con của hàm F.
 - Thân hàm không khai báo biến cục bộ nên không tạo thêm object nào.
- Phần thân chương trình: Chỉ chứa các lệnh (FOR, CALL, biểu thức...) và không tạo thêm object trong symbol table.

=> Kết quả: Symbol table chỉ in các đối tượng do người dùng khai báo, theo cấu trúc phân cấp scope. Các lời gọi trong thân chương trình như CALL WRITELN hay CALL WRITEI không được in ra vì WRITELN và WRITEI là built-in functions, và lệnh CALL không tạo object mới trong symbol table.

3. Kết quả thực hiện example3.kpl

- Mã nguồn của example3.kpl:

```
tests > example3.kpl

1  PROGRAM  EXAMPLE3;  (* TOWER OF HANOI *)
2  VAR      I:INTEGER;
3           N:INTEGER;
4           P:INTEGER;
5           Q:INTEGER;
6           C:CHAR;
7
8  PROCEDURE HANOI(N:INTEGER; S:INTEGER; Z:INTEGER);
9  BEGIN
10     IF N != 0 THEN
11     BEGIN
12         CALL HANOI(N-1,S,6-S-Z);
13         I:=I+1;
14         CALL WRITELN;
15         CALL WRITEI(I);
16         CALL WRITEI(N);
17         CALL WRITEI(S);
18         CALL WRITEI(Z);
19         CALL HANOI(N-1,6-S-Z,Z)
20     END
21 END;  (*END OF HANOI*)
22
23 BEGIN
24     FOR N := 1 TO 4 DO
25     BEGIN
26         FOR I:=1 TO 4 DO
27             CALL WRITEC(' ');
28             C := READC;
29             CALL WRITEC(C)
30         END;
31         P:=1;
32         Q:=2;
33         FOR N:=2 TO 4 DO
34         BEGIN
35             I:=0;
36             CALL HANOI(N,P,Q);
37             CALL WRITELN
38         END
39     END.  (* TOWER OF HANOI *)
40
```

- Kết quả thực hiện:

```
● → SymTab_Incompleted ./symtab tests/example3.kpl
Program EXAMPLE3
  Var I : Int
  Var N : Int
  Var P : Int
  Var Q : Int
  Var C : Char
  Procedure HANOI
    Param N : Int
    Param S : Int
    Param Z : Int

○ → SymTab_Incompleted
```

- **Giải thích kết quả thực hiện:**

- Các khai báo trong chương trình:

- **Khai báo chương trình:** PROGRAM EXAMPLE3; => Tạo đối tượng Program EXAMPLE3, đây là scope gốc của toàn bộ chương trình.
- **Khai báo các biến toàn cục:** Parser tạo 5 biến toàn cục: Var I: Int; Var N: Int; Var P: Int; Var Q: Int; Var C: Char.

=> Tất cả thuộc global scope của chương trình => được in với indent 4 spaces.

- **Khai báo thủ tục HANOI:** Parser thực hiện: Tạo đối tượng Procedure HANOI, thuộc scope chương trình -> Tạo 3 tham số (Param N, Param S và Param Z) -> Các tham số được đưa vào (danh sách tham số của HANOI và scope con) -> Thân thủ tục không có khai báo biến cục bộ nên không tạo thêm object nào.
=> Các tham số của HANOI thuộc scope con

=> được in với indent 8 spaces.

=> Phần thân chương trình: Chỉ chứa lệnh, vòng lặp, CALL..., không có khai báo hằng, biến, kiểu hay hàm mới => không tạo thêm object trong symbol table. Các lời gọi như CALL WRITELN, CALL WRITEI, CALL WRITEC... không được in vì chúng là lệnh, không phải khai báo object. Các thủ tục WRITELN, WRITEI, WRITEC, READC là built-in, nên cũng không được in ra trong symbol table.

4. Kết quả thực hiện example4.kpl

- Mã nguồn của example4.kpl:

```

tests > example4.kpl
1 PROGRAM EXAMPLE4; (* Example 4 *)
2 CONST MAX = 10;
3 TYPE T = INTEGER;
4 VAR A : ARRAY(. 10 .) OF T;
5     N : INTEGER;
6     CH : CHAR;
7
8 PROCEDURE INPUT;
9 VAR I : INTEGER;
10    TMP : INTEGER;
11 BEGIN
12     N := READI;
13     FOR I := 1 TO N DO
14         A(.I.) := READI;
15     END;
16
17 PROCEDURE OUTPUT;
18 VAR I : INTEGER;
19 BEGIN
20     FOR I := 1 TO N DO
21         BEGIN
22             CALL WRITEI(A(.I.));
23             CALL WRITELN;
24         END
25     END;
26
27 FUNCTION SUM : INTEGER;
28 VAR I : INTEGER;
29     S : INTEGER;
30 BEGIN
31     S := 0;
32     I := 1;
33     WHILE I <= N DO
34         BEGIN
35             S := S + A(.I.);
36             I := I + 1;
37         END
38     END;
39
40 BEGIN
41     CH := 'y';
42     WHILE CH = 'y' DO
43         BEGIN
44             CALL INPUT;
45             CALL OUTPUT;
46             CALL WRITEI(SUM);
47             CH := READC;
48         END
49     END. (* Example 4 *)

```

- Kết quả thực hiện:

```

● → SymTab_Incompleted ./symtab tests/example4.kpl
Program EXAMPLE4
  Const MAX = 10
  Type T = Int
  Var A : Arr(10,Int)
  Var N : Int
  Var CH : Char
  Procedure INPUT
    Var I : Int
    Var TMP : Int

  Procedure OUTPUT
    Var I : Int

  Function SUM : Int
    Var I : Int
    Var S : Int

○ → SymTab_Incompleted █

```

- **Giải thích kết quả thực hiện:**

- Chương trình bắt đầu với PROGRAM EXAMPLE4;, tạo scope gốc; Khai báo hằng CONST MAX = 10 tạo đối tượng MAX = 10 trong global scope; Khai báo kiểu TYPE T = INTEGER tạo kiểu T thay thế cho INTEGER, cũng thuộc global scope.
- Các biến toàn cục gồm: A là mảng 10 phần tử kiểu T (tức INTEGER), N là kiểu Int và CH là kiểu Char. Tất cả đều thuộc global scope và được in với indent 4 spaces.
- Thủ tục INPUT được parser tạo như một đối tượng thuộc scope chương trình. Bên trong thủ tục, parser tạo hai biến cục bộ I và TMP, đều kiểu Int. Không có tham số. Các biến cục bộ nằm trong scope con nên được thụt lề 8 spaces.
- Thủ tục OUTPUT cũng được parser tạo tương tự, nhưng chỉ có một biến cục bộ I kiểu Int.
- Phần thân chương trình chỉ chứa các câu lệnh (như WHILE, CALL, gán...), nên không phát sinh thêm đối tượng nào trong symbol table.

5. Kết quả thực hiện example5.kpl

- **Mã nguồn của example5.kpl:**

```
tests >  example5.kpl  
1  PROGRAM EXAMPLE5; (* Example 5 *)  
2  CONST C = 1;  
3  TYPE T = CHAR;  
4  FUNCTION F(I : INTEGER):CHAR;  
5  CONST B = C;  
6  TYPE A = ARRAY(.5.) OF T;  
7  BEGIN  
8  END;  
9  BEGIN  
10 END. (* Example 5 *)
```

- **Kết quả thực hiện:**

```
● → SymTab_Incompleted ./symtab tests/example5.kpl
Program EXAMPLE5
  Const C = 1
  Type T = Char
  Function F : Char
    Param I : Int
    Const B = 1
    Type A = Arr(5,Char)

○ → SymTab_Incompleted █
```

- **Giải thích kết quả thực hiện:**

- Khi parser đọc file, symbol table được xây dựng theo đúng thứ tự các khai báo xuất hiện.
- PROGRAM EXAMPLE5; tạo đối tượng chương trình EXAMPLE5, đây là scope gốc.
- CONST C = 1; tạo hằng C = 1 trong global scope.
- TYPE T = CHAR; tạo kiểu T ánh xạ sang CHAR, cũng thuộc global scope.
- Hàm F được khai báo với dạng FUNCTION F(I: INTEGER): CHAR;. Parser tạo đối tượng hàm F trong global scope, đồng thời tạo tham số I (kiểu Int) trong scope con của F.
- Bên trong hàm F, parser xử lý thêm:
 - Hằng cục bộ CONST B = C: B lấy giá trị từ C (C = 1 ở global), nên B được gán 1. B thuộc scope của F.
 - Kiểu cục bộ TYPE A = ARRAY(.5.) OF T: tạo kiểu A là mảng 5 phần tử của T (T = Char), thuộc scope F.
 - Thân hàm không có khai báo nên không tạo thêm object.

=> Phần thân chương trình cũng chỉ có lệnh, không có khai báo, nên không sinh thêm object trong symbol table.

6. Kết quả thực hiện example6.kpl

- Mã nguồn của example6.kpl:

```
tests > example6.kpl

1  PROGRAM EXAMPLE6;
2      CONST C1 = 10;
3          C2 = 'a';
4      TYPE T1 = ARRAY(. 10 .) OF INTEGER;
5      VAR V1 : INTEGER;
6          V2 : ARRAY(. 10 .) OF T1;
7
8      FUNCTION F(P1 : INTEGER; VAR P2 : CHAR) : INTEGER;
9      BEGIN
10         F := C1;
11     END;
12
13     PROCEDURE P(V1 : INTEGER);
14         CONST C1 = 'a';
15             C3 = 10;
16         TYPE T1 = INTEGER;
17             T2 = ARRAY(. 10 .) OF T1;
18         VAR V2 : T2;
19             V3 : CHAR;
20         BEGIN
21             V3 := 'a';
22             V1 := F(C3,V3);
23         END;
24
25 BEGIN
26     CALL P(C1);
27 END.
```

- Kết quả thực hiện:

```
● → SymTab_Incompleted ./symtab tests/example6.kpl
Program EXAMPLE6
  Const C1 = 10
  Const C2 = 'a'
  Type T1 = Arr(10,Int)
  Var V1 : Int
  Var V2 : Arr(10,Arr(10,Int))
  Function F : Int
    Param P1 : Int
    Param VAR P2 : Char

  Procedure P
    Param V1 : Int
    Const C1 = 'a'
    Const C3 = 10
    Type T1 = Int
    Type T2 = Arr(10,Int)
    Var V2 : Arr(10,Int)
    Var V3 : Char

○ → SymTab_Incompleted
```

- Giải thích kết quả thực hiện:

- Khi parser xử lý file example6.kpl, symbol table được xây dựng theo thứ tự khai báo và tuân theo cấu trúc phân cấp của các scope.
- PROGRAM EXAMPLE6; tạo đối tượng chương trình EXAMPLE6, là scope gốc.
- Các hằng toàn cục gồm C1 = 10 và C2 = 'a', đều thuộc global scope.
- Khai báo kiểu TYPE T1 = ARRAY(.10.) OF INTEGER; tạo kiểu T1 là mảng 10 phần tử kiểu Int.
- Các biến toàn cục gồm V1 : Int và V2 : Arr(10, Arr(10, Int)) (do T1 = Arr(10, Int)).
- Hàm F được khai báo với FUNCTION F(P1: INTEGER; VAR P2: CHAR): INTEGER;
- Parser tạo đối tượng hàm F (kiểu trả về Int), cùng hai tham số:
 - P1 : Int
 - P2 : Char (tham số VAR, truyền tham chiếu). Hàm F không có hằng, kiểu hay biến cục bộ nên scope của F chỉ chứa hai tham số.
- Thủ tục P được khai báo với PROCEDURE P(V1: INTEGER);.
- Parser tạo đối tượng thủ tục P cùng tham số V1 : Int.
- Bên trong P:
 - Hằng cục bộ: C1 = 'a' (che khuất C1 toàn cục) và C3 = 10
 - Kiểu cục bộ:
 - T1 = Int (do định nghĩa lại trong scope P)
 - T2 = Arr(10, Int)
 - Biến cục bộ:
 - V2 : Arr(10, Int) (dựa trên kiểu T2)
 - V3 : Char

=> Phần thân thủ tục và thân chương trình không có thêm khai báo nên không tạo đối tượng mới trong symbol table.

II. Kết quả thực hiện với các bài tập lý thuyết

Bài 1: Bài tập ma trận vuông. Viết chương trình ngôn ngữ KPL để nhập 1 ma trận vuông kích thước nxn, n nhập từ bàn phím, in ra 1 nếu ma trận là tam giác trên, in ra 0 nếu ngược lại.

- Mã nguồn:

```
tests > assignment1.kpl
1 PROGRAM UPPERTRIANGLE;
2 TYPE
3     ROW = ARRAY(. 100 .) OF INTEGER;
4 VAR
5     A: ARRAY(. 100 .) OF ROW;
6     N : INTEGER;
7     I : INTEGER;
8     J : INTEGER;
9     ISUPPER: INTEGER;
10
11 BEGIN
12     N := READI;
13
14     I := 1;
15     WHILE I <= N DO
16     BEGIN
17         J := 1;
18         WHILE J <= N DO
19         BEGIN
20             A(.I.)(.J.) := READI;
21             J := J + 1;
22         END;
23         I := I + 1;
24     END;
25
26     ISUPPER := 1;
27     I := 2;
28     WHILE I <= N DO
29     BEGIN
30         J := 1;
31         WHILE J < I DO
32         BEGIN
33             IF A(.I.)(.J.) != 0 THEN
34             BEGIN
35                 ISUPPER := 0;
36             END;
37             J := J + 1;
38         END;
39         I := I + 1;
40     END;
41
42     CALL WRITEI(ISUPPER);
43     CALL WRITELN;
44 END.
```

- **Kết quả thực hiện:**

```
● → SymTab_Incompleted ./symtab tests/assignment1.kpl
Program UPPERTRIANGLE
  Type ROW = Arr(100,Int)
  Var A : Arr(100,Arr(100,Int))
  Var N : Int
  Var I : Int
  Var J : Int
  Var ISUPPER : Int
○ → SymTab_Incompleted
```

- **Giải thích kết quả thực hiện:**

- Khi symtab phân tích chương trình KPL, nó chỉ ghi nhận các **khai báo**, không xử lý các lệnh như WHILE, IF, gán, READI, WRITEI... Vì vậy, bảng ký chỉ liệt kê những đối tượng xuất hiện trong vùng khai báo.
- Trong chương trình này, PROGRAM UPPERTRIANGLE; tạo đối tượng chương trình UPPERTRIANGLE, là scope gốc.
- Khai báo kiểu dữ liệu ở dòng 2–3 tạo kiểu ROW = Arr(100, Int), tức ROW là mảng 100 phần tử kiểu Integer.
- Các biến toàn cục gồm:
 - A là ma trận 100×100: A : ARRAY(.100.) OF ROW => A = Arr(100, Arr(100, Int)).
 - Các biến N, I, J, ISUPPER đều là Integer và thuộc global scope.

Do chương trình không khai báo FUNCTION hay PROCEDURE, symtab không tạo scope con và không có thêm đối tượng nào khác.

Bài 2: Viết chương trình tính và in ra tổng 2 số bằng ngôn ngữ KPL. Chỉ ra phân tích trái của chương trình (dãy số hiệu sản xuất được dùng trong suy dẫn trái).

- Mã nguồn:

```
tests > assignment2.kpl
1  PROGRAM SUMTWO NUMBERS;
2  VAR A: INTEGER; B: INTEGER; SUM: INTEGER;
3  BEGIN
4      A := READI;
5      B := READI;
6      SUM := a + b;
7      CALL WRITEI(SUM);
8      CALL WRITELN;
9  END.
10
```

- Kết quả thực hiện:

```
var ISUPPER : Int
● → SymTab_Incompleted ./symtab tests/assignment2.kpl
Program SUMTWO NUMBERS
  Var A : Int
  Var B : Int
  Var SUM : Int
○ → SymTab_Incompleted
```

- Giải thích kết quả thực hiện:

- PROGRAM SUMTWO NUMBERS; tạo đối tượng chương trình SUMTWO NUMBERS, đây là scope gốc.
- Ở dòng 2, chương trình khai báo ba biến toàn cục: A, B và SUM, đều có kiểu Int. Các biến này thuộc global scope nên được in với indent 4 spaces trong symbol table.
- Phần thân chương trình chỉ gồm các lệnh như READI, gán và WRITEI; vì chúng không phải là khai báo nên không được đưa vào symbol table.